



# KSFUPRO1KU – Functional Programming

## Lecture 1: Introduction and Getting Started

Niels Hallenberg

These slides are based on original slides by Michael R. Hansen, DTU. Thanks!!!



The original slides has been used at a course in functional programming at DTU.



# WELCOME to KSFUPRO1KU – Functional Programming

**Teacher:** Niels Hallenberg, [nh@itu.dk](mailto:nh@itu.dk)  
IT University of Copenhagen

**Teaching assistant:** Louise Holst Andersen (TA), [lohoan@itu.dk](mailto:lohoan@itu.dk)  
Halfdan Brage (TA), [halbra@itu.dk](mailto:halbra@itu.dk)  
Abdulrahman Mohsen (TA), [abdmoh@itu.dk](mailto:abdmoh@itu.dk)  
IT University of Copenhagen

**Homepage:** <https://learnit.itu.dk/course/view.php?id=3025457>



- Textbook: **Functional Programming using F#**  
by Michael R. Hansen and Hans Rischel.

ISBN: 9781107019027

Book homepage:

<http://www2.imm.dtu.dk/~mire/FSharpBook/>

Published on Cambridge University Press, May 2013

- Online ressource: F# for fun and profit,  
<https://fsharpforfunandprofit.com/books/>.
- F# is an **open-source** functional language integrated in the Visual Studio development platform and with access to all features in the .NET program library.
- You can use F# on all major platforms: Windows, Linux and Mac.
- Lectures, Thursday 08.15 – 10.00 in Aud 4 (4A60).
- Classes, Thursday 10.15 – 12.00 in 3A12 – 14.



Exam information is kept updated here:

<https://learnit.itu.dk/mod/wiki/create.php?wid=209&group&uid=12544&title=Exam>.

- **Date and Time:** 09:00 - 13:00, May 22, 2026
- Exam Syllabus: Functional Programming using F#, Michael R. Hansen and Hans Rischel, ISBN 9781107684065, Chapter 1 - 13.
- Selected posts on “F# for fun and profit”.
- **More to come . . .**

**You must pass the Mandatory assignments in order to attend exam.**

See course homepage in LearnIt for the details.

- Mandatory assignments do improve exam results!
- Main purpose of feedback on assignments is learning - not precise counting of points.
- If you are not satisfied with the grading, then just handin again.
- Remember the cut-off date - it is very important.
- **DO NOT USE AI SUPPORT.**



After this learning activity you should be able to:

- 1 apply and reflect on theories for modelling, analyzing and constructing functional declarative programs.
- 2 apply and reflect on the concepts behind functional programming compared to imperative and object oriented programming.
- 3 construct programs in F# and explain the basic principles behind functional programming using F#.
- 4 describe and explain solutions to problems in the context of functional programming.
- 5 apply core concepts of functional programming.
- 6 reason about the complexity of functional programs.



The subject of the course is functional, declarative programming in general and F# in particular. This includes the following themes:

- Functional Programming Paradigme:
  - first class functions
  - higher-order functions
  - type inference and polymorphism
  - recursion and tail-recursion
  - algebraic data types
  - strict and lazy evaluation
- Memory Management:
  - garbage collection
  - reference types
  - mutable versus immutable data
- Parallel Programming:
  - divide and conquer



## 1 Introduction

- Historic perspective

- Imperative models

- Object-oriented models

- Declarative models

- Functional programming background

## 2 Getting started with F#

- Using F#

- The interactive environment

- Declarations

- Recursion

- Types

## 3 Kattis



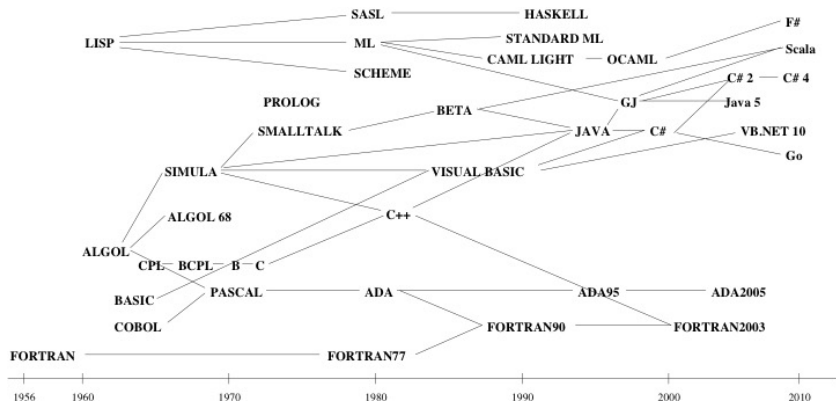
Url: [http://en.wikipedia.org/wiki/History\\_of\\_programming\\_languages](http://en.wikipedia.org/wiki/History_of_programming_languages)

- Lisp (concept, 1956, implementation 1959)
- ML (1973)
- Scheme (1975)
- Standard ML (1984)
- Common LISP (1984)
- Miranda (1986)
- Erlang (1987)
- Haskell (1990)
- Java (1995)
- C# (2000)
- F# (2005)
- Scala (2004, first release)

Url: [http://en.wikipedia.org/wiki/Timeline\\_of\\_programming\\_languages](http://en.wikipedia.org/wiki/Timeline_of_programming_languages)



# The Diagram





- **Imperative models of computations** are expressed in terms of **states** and sequences of **state-changing operations**

Example:

```
int i = 0;
int s = 0;
int[] a = {1, 2, 3, 4, 5};

while (i < a.length) {
    s += a[i];
    i += 1;
}
```

An imperative model describes *how* a solution is obtained



- An **object** is characterized by a **state** and an **interface** specifying a collection of **state-changing operations**.
- **Object-oriented models of computations** are expressed in terms of a collection of objects which exchange messages by using interface operations.

Object-oriented models add structure to imperative models

An object-oriented model describes *how* a solution is obtained



In declarative models focus is on *what* a solution is.

- Functional programming
  - A program is expressed as a mathematical function

$$f : A \rightarrow B$$

and evaluations of function applications guide computations.

Example:

```
// fact(0) = 1
// fact(n) = n * fact(n - 1) [if n > 0]

let rec fact n =
  match n with
  | 0 -> 1
  | _ -> n * fact(n-1)
```



## Some advantages

- fast prototyping based on abstract concepts
- easier reasoning about the smaller features (functions) to build larger features (application of functions).
- easier to parallelize on multi-core platforms.
- easier to compose functions.
- less syntax, easier to read.

F# is as efficient as C#



In **functional programming**, the model of computation is the application of functions to arguments. **no side-effects**

- Introduction of  **$\lambda$ -calculus** around 1930 by Church and Kleene when investigating function definition, function application, recursion and computable functions. For example,  $f(x) = x + 2$  is represented by  $\lambda x.x + 2$ .
- Introduction of the type-less functional-like programming language LISP was developed by McCarthy in the late 1950s.
- Introduction of functional languages with a strong type system like ML (by Milner) and Miranda (by Turner) in the 1970s.

# Some background of the “SML-family”



- **Standard Meta Language** (SML) was originally designed for theorem proving  
Logic for Computable Functions (Edinburgh LCF)  
Gordon, Milner, Wadsworth (1977)
- High quality compilers, e.g. **Standard ML of New Jersey** and **Moscow ML**, based on a *formal semantics*  
Milner, Tofte, Harper, MacQueen 1990 & 1997
- SML-like systems (SML, OCAML, **F#**, ...) have now applications far away from its origins  
Compilers, Artificial Intelligence, Web-applications, Financial sector, ...
- **F#** is integrated in the .net environment
- Declarative aspects are sneaking into more "main stream languages"
- Often used to teach high-level programming concepts



Teach **abstraction** (not a concrete programming language)

- Modelling
- Design
- Programming

Why?

More complex problems can be solved in an succinct, elegant and understandable manner

How?

Solving a broad class of problems showing the applicability of the theory, concepts, techniques and tools.





Functional programming techniques once mastered are useful for the design of programs in other programming paradigms as well.

Mastering functional programming makes you a better developer.

A developer should not be constrained by languages nor paradigms.



In a given programming language design, a first-class object is an entity which supports all the operations generally available to other entities.

These operations typically include being *passed as an argument*, *returned from a function*, and *assigned to a variable*.

Wikipedia:

[https://en.wikipedia.org/wiki/First-class\\_citizen](https://en.wikipedia.org/wiki/First-class_citizen)



- Functions as first class citizens
- Structured values like lists, trees, . . .
- Strong and flexible type discipline, including  
    type inference and polymorphism
- Imperative and object-oriented programming  
    assignments, loops, arrays, objects, Input/Output, etc.  
    In this course, we avoid imperative features.

## Programming as a modelling discipline

- High-level programming, declarative programming
- Fast prototyping



## 1 Introduction

- Historic perspective

- Imperative models

- Object-oriented models

- Declarative models

- Functional programming background

## 2 Getting started with F#

- Using F#

- The interactive environment

- Declarations

- Recursion

- Types

## 3 Kattis



- F# is an open source functional language integrated with Visual Studio development platform.
- Runs on .NET platform.
- Available on all major systems, Windows, Linux and Mac:  
<https://fsharp.org>.



```
2*3 + 4;;
```

⇐ Input to the F# system

```
val it : int = 10
```

⇐ Answer from the F# system

- The *keyword* `val` indicates a value is computed
- The *integer* `10` is the computed value
- `int` is the *type* of the computed value
- The *identifier* `it` names the (last) computed value

The notion *binding* explains which entities are named by identifiers.

`it` ↦ `10`      reads: “variable `it` is bound to 10”



A value declaration has the form: `let identifier = expression`

```
let price = 25 * 5;;
```

⇐ A declaration as input

```
val price : int = 125
```

⇐ Answer from the F# system

The effect of a declaration is a binding: `price ↦ 125`

Bound identifiers can be used in expressions and declarations, e.g.

```
let newPrice = 2*price;;
```

```
val newPrice : int = 250
```

```
newPrice > 500;;
```

```
val it : bool = false
```

A collection of bindings

|          |   |       |
|----------|---|-------|
| price    | ↦ | 125   |
| newPrice | ↦ | 250   |
| it       | ↦ | false |

is called an environment



```
int price;  
price = 25*5;  
  
int newPrice;  
newPrice = 2*price;  
  
boolean b = newPrice > 500;
```





Declaration of the circle area function:

```
let circleArea r = System.Math.PI * r * r;;
```

- `System.Math` is a program library
- `PI` is an identifier (with type `float`) for  $\pi$  in `System.Math`

The type is **automatically inferred** in the answer:

```
val circleArea : float -> float
```

A function type:  $\langle \text{argument type} \rangle \rightarrow \langle \text{return type} \rangle$ .

Applications of the function:

```
circleArea 1.0;; (* this is a comment *)  
val it : float = 3.141592654
```

```
circleArea(3.2);; // A comment: optional brackets  
val it : float = 32.16990877
```

# Anonymous functions: by example (1)



The keyword `fun` declares an *anonymous* function, i.e., a function without name.

```
> fun r -> System.Math.PI * r * r;;  
val it: r: float -> float  
  
> it 2.0;;  
val it: float = 12.56637061  
  
> (fun r -> System.Math.PI * r * r) 2.0;;  
val it: float = 12.56637061
```

In Java:

```
import java.util.function.DoubleUnaryOperator;  
  
public static DoubleUnaryOperator mkCircleArea()  
{  
    return r -> Math.PI * r * r;  
}  
  
mkCircleArea().applyAsDouble(2.0);
```



A function declaration is an expression, i.e., the expression evaluates to a function.

Below functional expression converts booleans to integers.

```
fun b ->
  match b with
  | true -> 1
  | false -> 0
```

```
val it: b: bool -> int
```

In Java:

```
import java.util.function.ToIntFunction;

public static ToIntFunction<Boolean> mkBoolToInt ()
{
  return b -> b?1:0;
}

mkBoolToInt().applyAsInt(true);
mkBoolToInt().applyAsInt(false);
```



Below functional expression converts integers to booleans.

```
fun i ->
  match i with
  | 0 -> true
  | 1 -> false
```

```
val it: i: int -> bool
```

**Why is above function problematic?**

Does below Java method have same problem?

```
import java.util.function.IntPredicate;

public static IntPredicate mkIntToBool() {
  return i -> (i==0)?false:true;
}

mkIntToBool().test(0);
mkIntToBool().test(1);
```



Below function has same semantics as the Java method:

```
fun i ->
  match i with
  | 0 -> true
  | _ -> false
```

```
val it: i: int -> bool
```

The parameter `i` is only used to pass into `match` expression.

The keyword `function` reduces syntax for this special case.

```
function
  | 0 -> true
  | _ -> false
```

```
val it: _arg1: int -> bool
```



An **anonymous function** computing the number of days in a month:

```
function
| 1 -> 31 // January
| 2 -> 28 // February // not a leap year
| 3 -> 31 // March
| 4 -> 30 // April
| 5 -> 31 // May
| 6 -> 30 // June
| 7 -> 31 // July
| 8 -> 31 // August
| 9 -> 30 // September
| 10 -> 31 // October
| 11 -> 30 // November
| 12 -> 31;; // December
... warning ... Incomplete pattern matches ...
val it : int -> int = <fun:clo@17-2>

it 2;;
val it : int = 28
```

**A functional expression with a pattern for every month**



One *wildcard pattern* `_` can cover many similar cases:

```
function
| 2  -> 28  // February
| 4  -> 30  // April
| 6  -> 30  // June
| 9  -> 30  // September
| 11 -> 30  // November
| _  -> 31;; // All other months
```

An even more succinct definition can be given using an *or-pattern*:

```
function
| 2          -> 28  // February
| 4|6|9|11  -> 30  // April, June, September, November
| _          -> 31  // All other months
;;
```



One *wildcard pattern* `_` can cover many similar cases:

```
function
| 2  -> 28  // February
| 4  -> 30  // April
| 6  -> 30  // June
| 9  -> 30  // September
| 11 -> 30  // November
| _  -> 31;; // All other months
```

An even more succinct definition can be given using an *or-pattern*:

```
function
| 2          -> 28  // February
| 4|6|9|11 -> 30  // April, June, September, November
| _          -> 31  // All other months
;;
```



# Recursion. Example $n! = 1 \cdot 2 \cdot \dots \cdot n$ , $n \geq 0$



Mathematical definition:

recursion formula

$$\begin{aligned} 0! &= 1 & (i) \\ n! &= n \cdot (n-1)!, \quad \text{for } n > 0 & (ii) \end{aligned}$$

Computation:

$$\begin{aligned} &3! \\ = &3 \cdot (3-1)! & (ii) \\ = &3 \cdot 2 \cdot (2-1)! & (ii) \\ = &3 \cdot 2 \cdot 1 \cdot (1-1)! & (ii) \\ = &3 \cdot 2 \cdot 1 \cdot 1 & (i) \\ = &6 \end{aligned}$$



Function declaration:

```
let rec fact = function
  | 0 -> 1                (* i *)
  | n -> n * fact(n-1);;  (* ii *)
val fact : int -> int
```

Evaluation:

```
fact(3)
~> 3 * fact(3 - 1)      (ii) [n ↦ 3]
~> 3 * 2 * fact(2 - 1)  (ii) [n ↦ 2]
~> 3 * 2 * 1 * fact(1 - 1) (ii) [n ↦ 1]
~> 3 * 2 * 1 * 1        (i)  [n ↦ 0]
~> 6
```

$e_1 \rightsquigarrow e_2$  reads:  $e_1$  evaluates to  $e_2$



A recursive implementation of `fact` in Java:

```
public static int fact(int n) {  
    if (n==0)  
        return 1;  
    else  
        return n * fact(n-1);  
}
```

An imperative implementation of `fact` in Java:

```
public static int factImperative(int n) {  
    int res = 1;  
    for (int i=2; i<=n; i++)  
        res *= i;  
    return res;  
}
```

## Recursion. Example $x^n = x \cdot \dots \cdot x$ , $n$ occurrences of $x$



Mathematical definition:

recursion formula

$$x^0 = 1 \quad (1)$$

$$x^n = x \cdot x^{n-1}, \quad \text{for } n > 0 \quad (2)$$

Function declaration:

```
let rec power = function
  | (_, 0) -> 1.0                (* 1 *)
  | (x, n) -> x * power(x, n-1) (* 2 *)
```

```
val power: float * int -> float
```

Patterns:

$(-, 0)$  matches any **pair** of the form  $(x, 0)$ .

The **wildcard** pattern  $_$  matches any value.

$(x, n)$  matches any pair  $(u, i)$  **yielding** the bindings

$x \mapsto u, n \mapsto i$



## Function declaration:

```
let rec power = function
  | (_, 0) -> 1.0                (* 1 *)
  | (x, n) -> x * power(x, n-1)  (* 2 *)
```

## Evaluation:

|                                                           |                                          |
|-----------------------------------------------------------|------------------------------------------|
| <code>power(4.0, 2)</code>                                |                                          |
| $\rightsquigarrow 4.0 * \text{power}(4.0, 2 - 1)$         | Clause 2, $[x \mapsto 4.0, n \mapsto 2]$ |
| $\rightsquigarrow 4.0 * \text{power}(4.0, 1)$             |                                          |
| $\rightsquigarrow 4.0 * (4.0 * \text{power}(4.0, 1 - 1))$ | Clause 2, $[x \mapsto 4.0, n \mapsto 1]$ |
| $\rightsquigarrow 4.0 * (4.0 * \text{power}(4.0, 0))$     |                                          |
| $\rightsquigarrow 4.0 * (4.0 * 1)$                        | Clause 1                                 |
| $\rightsquigarrow 16.0$                                   |                                          |



A recursive implementation of `power` in Java:

```
public static double power(double x, int n) {  
    if (n==0)  
        return 1.0;  
    else  
        return x * power(x,n-1);  
}
```

An imperative implementation of `fact` in Java:

```
public static double powerImperative(double x, int n) {  
    double res = 1.0;  
    for (int i=1; i<=n; i++)  
        res *= x;  
    return res;  
}
```



Form:

if  $b$  then  $e_1$  else  $e_2$

Evaluation rules:

if **true** then  $e_1$  else  $e_2 \rightsquigarrow e_1$

if **false** then  $e_1$  else  $e_2 \rightsquigarrow e_2$

Alternative declarations:

```
let rec fact n      = if n=0 then 1
                      else n * fact(n-1);;
```

```
let rec power(x,n)  = if n=0 then 1.0
                      else x * power(x,n-1);;
```

Use of patterns often give more understandable programs



Type name `bool`

Values `false`, `true`

| Operator         | Type                         |          |
|------------------|------------------------------|----------|
| <code>not</code> | <code>bool -&gt; bool</code> | negation |

|                                                                |
|----------------------------------------------------------------|
| <code>not true = false</code><br><code>not false = true</code> |
|----------------------------------------------------------------|

Expressions

`e1 && e2`      “conjunction  $e_1 \wedge e_2$ ”

`e1 || e2`      “disjunction  $e_1 \vee e_2$ ”

— are lazily evaluated (short circuit eval.), e.g.

|                                                           |
|-----------------------------------------------------------|
| <code>1 &lt; 2    5 / 0 = 1</code><br><code>↪ true</code> |
|-----------------------------------------------------------|

Precedence: `&&` has higher than `||`





Type name `string`

Values `"abcd"`, `" "`, `" "`, `"123\"321"` (escape sequence for `"`)

| Operator                      | Type                                    |                  |
|-------------------------------|-----------------------------------------|------------------|
| <code>String.length</code>    | <code>string -&gt; int</code>           | length of string |
| <code>+</code>                | <code>string*string -&gt; string</code> | concatenation    |
| <code>= &lt; &lt;= ...</code> | <code>string*string -&gt; bool</code>   | comparisons      |
| <code>string</code>           | <code>obj -&gt; string</code>           | conversions      |

## Examples

```
- "auto" < "car";  
> val it = true : bool  
  
- "abc"+"de";  
> val it = "abcde": string
```

```
- String.length("abc"^"def");  
> val it = 6 : int  
  
- string(6+18);  
> val it = "24": string
```



A *tuple* has a finite number of elements, e.g.,  $(e_1, \dots, e_n)$ :

```
> (42, 5);;  
val it: int * int = (42, 5)  
  
> (34.0, "A string");;  
val it: float * string = (34.0, "A string")  
  
> ("and why not a few functions", fact, power);;  
val it: string * (int -> int) * (float * int -> float) =  
  ("and why not a few functions",  
   <fun:it@114-2>, <fun:it@114-3>)
```

A *pair* is a special case tuple with two elements,  $(e_1, e_2)$ .

Functions are first-class citizens.

Functions can be used as data, similar to a string or integer.

Functions can be passed as argument to a function.

# Types — every expression has a type $e : \tau$



Basic types:

|          | type name | example of values |
|----------|-----------|-------------------|
| Integers | int       | ~27, 0, 15, 21000 |
| Floats   | float     | ~27.3, 0.0, 48.21 |
| Booleans | bool      | true, false       |

Pairs:

If  $e_1 : \tau_1$  and  $e_2 : \tau_2$

then  $(e_1, e_2) : \tau_1 * \tau_2$

pair (tuple) type constructor

Functions:

if  $f : \tau_1 \rightarrow \tau_2$  and  $a : \tau_1$

then  $f(a) : \tau_2$

function type constructor

Examples:

```
(4.0, 2): float*int
power: float*int -> float
power(4.0, 2): float
```

\* has higher precedence than  $\rightarrow$



```
let rec power = function
  | (_, 0) -> 1.0 (* 1 *)
  | (x, n) -> x * power(x, n-1) (* 2 *)
```

- The type of the function must have the form:  $\tau_1 * \tau_2 \rightarrow \tau_3$ , because argument is a pair.
- $\tau_3 = \text{float}$  because `1.0:float` (Clause 1, function value.)
- $\tau_2 = \text{int}$  because `0:int`.
- `x*power(x, n-1):float`, because  $\tau_3 = \text{float}$ .
- multiplication can have  
`int*int -> int` or `float*float -> float`  
as types, but no “mixture” of `int` and `float`
- Therefore `x:float` and  $\tau_1 = \text{float}$ .

The F# system determines the type `float*int -> float`



- The interactive environment
- Values, expressions, types, patterns
- Declarations of values and recursive functions
- Binding, environment and evaluation
- Type inference



## 1 Introduction

- Historic perspective

- Imperative models

- Object-oriented models

- Declarative models

- Functional programming background

## 2 Getting started with F#

- Using F#

- The interactive environment

- Declarations

- Recursion

- Types

## 3 Kattis



Kattis offers exercises at all levels.

Great way to exercise programming, including F#

Example exercises:

- <https://open.kattis.com/problems/hello>
- <https://open.kattis.com/problems/hipphipp>
- <https://open.kattis.com/problems/timeloop>
- <https://open.kattis.com/problems/reduplikation>